

PROJECT

4

BUILD A CHATBOT WITH AMAZON LEX

CHATBOT SERIES

Prepared by:
Amrize JK
Certified Solutions Architect Associate

Objective

The AWS Chatbot Series helps learners build intelligent voice/text-based bots using Amazon Lex. It covers creating intents, custom slots, and integrating with AWS Lambda for dynamic responses.

This hands-on series builds a mini Banking Chatbot, teaching both basic and advanced setups like fallback handling, context carryover and user data storage with real world relevance.

Services

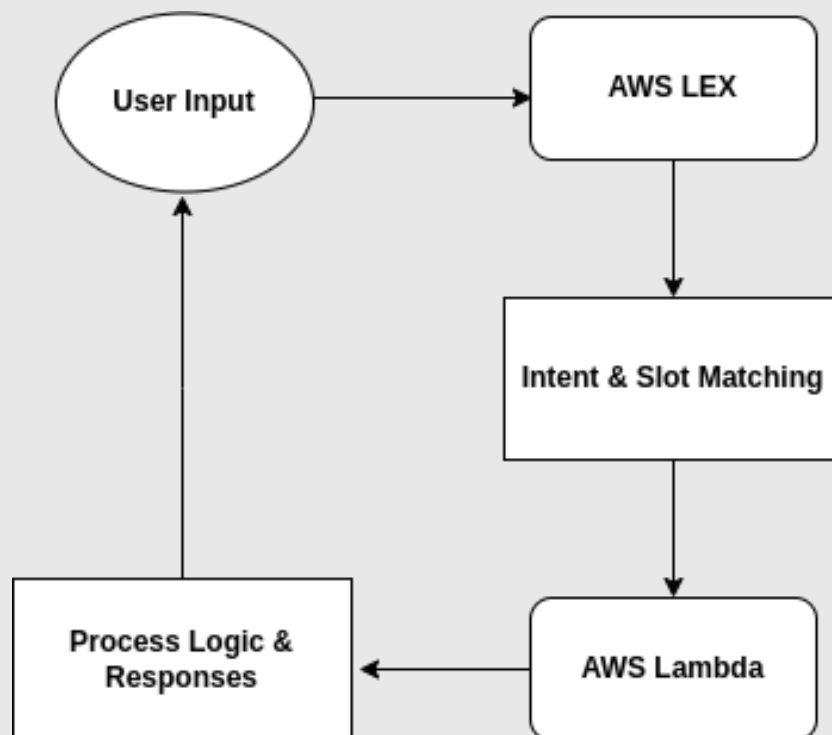
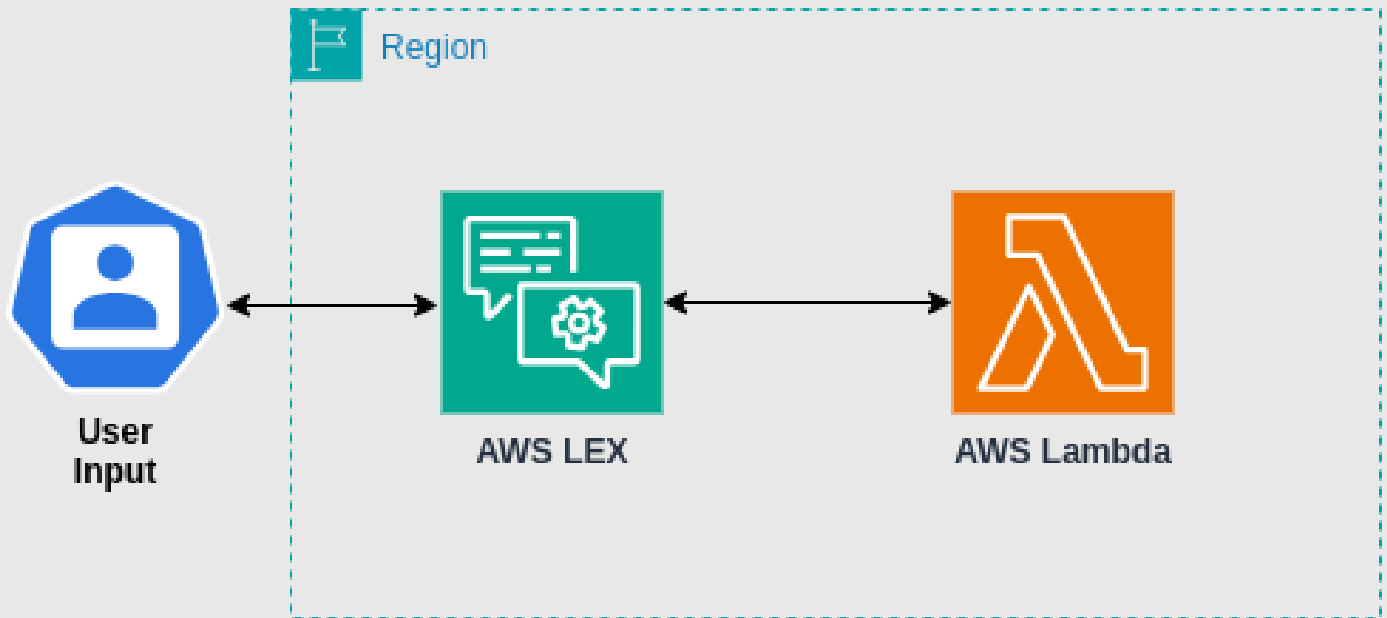
Amazon Lex - For building and deploying conversational interfaces

AWS Lambda - For backend logic and real-time integration with Lex.

Amazon CloudWatch - For monitoring chatbot and Lambda interactions.

IAM (Identity and Access Management) - To manage permissions for Lambda and Lex.

Architecture Diagram



The user interacts with Amazon Lex, which identifies the intent and extracts slot values. Lex then triggers an AWS Lambda function to process the logic and return a response, which is delivered back to the user. Features like custom slots, context carryover, and fallbacks enhance the chatbot experience.

Steps to Implement 1

Build a Chatbot with Amazon Lex

1. Set Up Your Lex Chatbot:

Go to the Amazon Lex console and click on “Create bot” to begin building your chatbot. Choose “Create from scratch,” enter a bot name (e.g., BankerBot), and select the default language like English (US). Enable voice interaction and choose a voice ID if you want audio responses. Click “Create” to launch your chatbot and proceed to the intent configuration section.

2. Create Your First Intent:

Click “Add intent” → “Create intent” to define your bot’s first action. Name the intent (e.g., CheckBalanceIntent) and add sample utterances like “What’s my balance?” or “Check account balance.” Add slot types if your intent needs more details like account type. Set a response message like “Sure, I can help with that!” and click “Save Intent” to lock in your changes.

3. Manage FallbackIntent:

Select the default FallbackIntent created automatically by Lex. Edit its response to guide users when their input doesn’t match any known intents. Use messages like “Sorry, I didn’t catch that. Can you rephrase?” to keep the conversation smooth. Add a few variations to improve user experience and click “Save Intent.”

Steps to Implement 2

Add Custom Slots to a Lex Chatbot

1. Navigate to Slot Types and Create One:

In the Amazon Lex console, open your chatbot's dashboard and select Slot Types from the left-hand menu. Click on Create slot type to begin setting up a custom slot that will help Lex identify specific user inputs more accurately. This step prepares the bot to understand terms like different bank account types in the conversation.

2. Define AccountType Slot:

Name your new slot type as AccountType to indicate it will handle types of bank accounts. Then, enter values such as Savings, Checking, and Credit to represent common options users might mention. These values teach Lex how to categorize the input, and once done, click Save to store the slot type for use in intents.

3. Create the CheckBalance Intent:

Go to Intents and click Add intent to create one named CheckBalanceIntent. Add sample utterances like "Check my savings account balance." Attach the AccountType slot, define a response like "Checking your {AccountType} account balance," and build the bot.

Steps to Implement 3

Connect Amazon Lex with Lambda

1. Create Your AWS Lambda Function:

Go to the AWS Lambda console and click “Create function.” Choose “Author from scratch,” enter a function name like BankerBotLambda, and select a runtime such as Python or Node.js. Write the code to process the chatbot’s intents and slot data, then save and deploy the function.

2. Connect AWS Lambda with Amazon Lex:

In the Amazon Lex console, open your chatbot and select the intent you want to connect (e.g., CheckBalanceIntent). Under the Fulfillment section, choose “AWS Lambda function” as the fulfillment method and select your deployed Lambda function from the list. Save the intent settings.

3. Connect Your CheckBalance Intent with Your Lambda Function:

Ensure your Lambda function correctly handles the intent and processes slot information like account types. Test the intent in the Lex test window to confirm the Lambda invocation returns the expected responses. Finally, rebuild your bot to apply all the changes.

Steps to Implement 4

Save User Info with a Lex Chatbot

1. Create an Output Context from CheckBalance:

Open your CheckBalanceIntent in the Amazon Lex console. Under the Output Contexts section, create a new context (e.g., CheckBalanceContext) and set its lifespan to maintain the conversation state. This context will hold user information to be used in follow-up intents.

2. Create the FollowupCheckBalance Intent:

Add a new intent called FollowupCheckBalanceIntent. Define sample utterances that a user might say as a follow up, like “What about my savings account?” or “And my checking balance?”. Set the input context to the output context created earlier (CheckBalanceContext) to ensure context carryover.

3. Implement Context Carryover:

Configure your chatbot so that information from the first intent (like account type) is passed to the follow up intent using the context. Connect the follow-up intent to your Lambda function to handle logic based on the carried-over context. Save and build your bot to activate context-aware conversations.

Steps to Implement 5

Set Up Multiple Slots in a Lex Chatbot

1. Create the TransferFunds Intent:

In Lex, create TransferFundsIntent with utterances like “Transfer ₹1000 from savings to checking.” Add multiple required slots like Amount, FromAccountType, and ToAccountType.

2. Use Lex Features:

Add prompt variations, slot prompts, and confirmation messages to guide users and make the chatbot interaction smoother and dynamic.

3. Connect to AWS Lambda:

Set Lambda as the fulfillment method. Ensure your function handles multiple slot values, processes the logic, and returns responses.

4. Enable Context Carryover:

Create an output context in TransferFundsIntent and link it to follow-up intents. This keeps conversation data flowing across steps.

5. Clean Up Resources:

Once done, delete the chatbot, slots, and Lambda functions from the AWS Console to avoid extra usage and keep things organized.

Story Time

Amazon Lex (The Talkative Spellcaster):

Amazon Lex is your friendly spellcaster who loves chatting with humans. Give it some magical phrases (intents and utterances), and it'll understand what you're saying like a wise wizard. Whether it's a greeting or a treasure hunt request, Lex is ready with a clever reply!

AWS Lambda (The Invisible Helper Elf):

Lambda is your silent elf working behind the scenes. Whenever the spellcaster (Lex) needs help, Lambda pops in, runs around to fetch or process information, and disappears without a trace no servers, no fuss, just pure magic!

Custom Slots (The Magic Word Catchers):

These are little spell traps inside Lex that catch important words from your sentence like catching "savings" from "check my savings balance." You teach them what words to catch, and they remember like loyal familiars.

Context Carryover (The Memory Potion):

Give your bot a memory potion with context carryover! It lets the chatbot remember your last question, so it doesn't feel like you're talking to a forgetful goblin. It links conversations like a smart storyteller.

Intents (The Spellbook Pages):

Each intent is a special page in your bot's spellbook. You define what a user might say and what your bot should do in return. From "CheckBalance" to "TransferFunds," every page holds a magical interaction!

BY
AMRIZE J.K

THANK YOU!

